



Тема 3.4. Работа с репозиторием кода через ИИ

Задача занятия

Сегодня я покажу, как исследовать существующий репозиторий до изменения кода. В центре занятия будет не синтаксис Python, а доказательная работа с архитектурой: мы найдём границу API, восстановим направление зависимостей, отделим реальные вызовы от случайных текстовых совпадений, обратимся к истории Git и на основании проверенных фактов подготовим план миграции.

Я буду работать с проектом `hermes-multiagent`, который мы создали во втором модуле. Внутри него уже существует учебный платёжный сценарий с действующим legасу-клиентом и подготовленным клиентом новой версии, поэтому нам не придётся придумывать искусственный пример отдельно от репозитория.

Определяем рабочую область

Пометка преподавателю. В Visual Studio Code откройте каталог проекта `hermes-multiagent` как корневую папку рабочего пространства.

Я начинаю именно с корня проекта, потому что относительный путь имеет смысл только внутри известной рабочей области. ИИ-ассистент, терминал и редактор должны смотреть на один и тот же каталог, иначе одинаковое имя файла может относиться к разным объектам, а поиск даст неполную картину.

Навигация по кодовой базе начинается не с последовательного чтения всех файлов. Я сначала определяю интересующую меня архитектурную границу, затем расширяю область поиска только по обнаруженным зависимостям. Такой подход удерживает контекст небольшим и не позволяет случайным совпадениям увести исследование в другой модуль.

Пометка преподавателю. В проводнике Visual Studio Code раскройте каталог `demo_codebase`, затем `payments` и откройте файл `gateway.py`.

Я рассматриваю этот файл как границу взаимодействия с платёжной системой. Для архитектурного анализа сейчас важен сам факт, что в одном модуле объявлен старый клиент и рядом с ним уже существует клиент новой версии. Реализацию возвращаемого словаря я пока не разбираю построчно, потому что сначала должен понять место этого модуля в общей системе.

Публичный контракт API определяется не только именем класса. Сигнатура метода задаёт форму входных данных, а наблюдаемый результат задаёт обязательство перед вызывающим кодом. В текущем проекте старый и новый клиенты принимают одинаковые данные, но используют разные имена методов и по-разному обозначают источник платежа в результате. Этого уже достаточно, чтобы считать миграцию изменением поведения, даже если структура ответа в остальном похожа.

Я не делаю вывод, что новый клиент уже используется, только потому, что его класс присутствует в репозитории. Объявление возможности и включение этой возможности в рабочий маршрут — разные состояния системы. Теперь мне нужно найти место, где выбирается конкретная реализация.

Находим точку привязки к API

Пометка преподавателю. Откройте файл `demo_codebase/payments/service.py` и установите курсор на строку импорта платёжного клиента.

Здесь я вижу статическую зависимость сервисного слоя от legacy-класса. Импорт связывает модуль с конкретной реализацией ещё до выполнения программы, поэтому замена API неизбежно затронет эту точку. При этом сам файл выполняет роль адаптера: остальная система обращается к функции обработки платежа и не обязана напрямую знать класс шлюза.

Сервисный слой полезен как граница миграции. Если вызывающий код зависит от функции `process_payment`, а выбор клиента скрыт внутри неё, то новую реализацию можно подключить локально. Чем меньше модулей знают конкретный класс внешнего клиента, тем меньше радиус изменения.

Пометка преподавателю. В терминале из корня проекта выполните точный поиск старого класса.

```
rg -n "LegacyPaymentGateway" demo_codebase tests
```

Я использую текстовый поиск как первый способ получить кандидатов на анализ, но не считаю каждое совпадение зависимостью. В результате видны объявления класса и обращения из сервисного слоя. Отдельное совпадение находится в тестовом материале для инструмента поиска, поэтому его назначение нужно проверить до включения в план миграции.

Номер строки в выводе нужен не для отчётности. Он превращает ответ ИИ или поисковой команды в проверяемую ссылку, по которой я могу сразу открыть место и подтвердить смысл совпадения. Если ассистент сообщает только имя файла без позиции и контекста, такой ответ остаётся гипотезой.

Пометка преподавателю. В терминале выполните поиск старого метода.

```
rg -n "charge_legacy" demo_codebase tests
```

Теперь я отделяю зависимость от класса от зависимости от его метода. В текущем проекте рабочий вызов старого метода сосредоточен в `service.py`, поэтому смена имени операции не распространяется на модуль оформления заказа. Это подтверждает, что сервис действительно изолирует внешний API.

Поиск по имени метода также защищает от неполной миграции. Простая замена импорта может оставить старый вызов и привести к ошибке только во время выполнения. План миграции должен учитывать не файл целиком, а все места, где контракт старого API проявляется в рабочем маршруте.

Восстанавливаем направление зависимости

Пометка преподавателю. Откройте файл `demo_codebase/orders/checkout.py`.

Я вижу потребителя платёжного сервиса, а не потребителя платёжного шлюза. Модуль оформления заказа вызывает `process_payment` и принимает результат через устойчивую внутреннюю границу. Это означает, что направление зависимости выглядит как переход от оформления заказа к сервису оплаты, а затем от сервиса к конкретному шлюзу.

Архитектурно этот маршрут можно записать так:

```
checkout.py → service.py → gateway.py
```

Стрелка показывает знание одного модуля о другом. Обратной зависимости здесь нет, поэтому замена нижнего звена не должна требовать изменения верхнего звена, пока сохраняется контракт функции `process_payment`. Именно направление зависимостей, а не близость файлов в дереве проекта, определяет ожидаемый радиус миграции.

Пометка преподавателю. В терминале выполните поиск функции `process_payment` в рабочем коде.

```
rg -n "process_payment" demo_codebase
```

Результат показывает определение функции и её прикладного потребителя. Для текущего репозитория это важнее общего количества файлов: мы нашли точку входа в платёжный сценарий и убедились, что внешний модуль не создаёт gateway самостоятельно.

Если бы поиск показал прямое создание платёжного клиента в нескольких прикладных модулях, план пришлось бы расширить. Здесь такой необходимости нет. Я фиксирую `service.py` как точку замены, а `checkout.py` как место проверки наблюдаемого эффекта.

Используем ИИ для построения проверяемой гипотезы

Пометка преподавателю. Откройте новый чат ИИ-ассистента в контексте проекта и отправьте следующий запрос.

```
Исследуй путь вызова process_payment в текущем проекте. Построй одну цепочку от прикладного потребителя до конкретного платёжного клиента, не изменяй файлы и для каждого перехода укажи путь и номер строки, которые можно проверить вручную.
```

Я формулирую запрос так, чтобы получить один архитектурный результат, а не общий пересказ репозитория. Требование указать пути и строки заставляет ассистента связать вывод с наблюдаемыми файлами. Запрет на изменение отделяет исследование от реализации: на этом этапе мне нужна карта зависимости, а не преждевременный patch.

Ответ ИИ не становится фактом автоматически. Я сопоставляю предложенную цепочку с уже открытыми файлами и результатами `rg`. Если ассистент добавляет несуществующего потребителя или пропускает `service.py`, я исправляю карту по данным репозитория, а не подгоняю репозиторий под ответ модели.

ИИ особенно полезен для объяснения связи между найденными местами. Текстовый поиск находит совпадения, но сам не решает, какое из них является объявлением, какое входит в рабочий маршрут и какое существует только как тестовый материал. Эту классификацию ассистент может предложить, однако подтверждение всё равно выполняется открытием конкретного файла.

Отделяем рабочее использование от совпадения в тесте

Пометка преподавателю. Откройте найденное поиском место с `LegacyPaymentGateway` в файле `tests/test_dev_tools.py`.

Здесь имя `legasu`-класса используется как строка для проверки поискового инструмента. Такая строка не создаёт объект, не импортирует класс и не входит в платёжный маршрут. Если механически считать все результаты `rg` точками миграции, этот тест попадёт в план без архитектурного основания.

Я называю это текстовым совпадением, а не исполняемой зависимостью. Оно может остаться актуальным после миграции, если тест по-прежнему проверяет способность инструмента находить заданный шаблон. Решение об изменении такого места принимается по назначению теста, а не по совпадению имени.

Пометка преподавателю. В терминале выполните поиск нового класса.

```
rg -n "PaymentGatewayV2" demo_codebase tests
```

В текущем проекте новый класс объявлен в `gateway.py` и напрямую вызывается в тесте, но не подключён к `process_payment`. Я получаю важный промежуточный вывод: целевой API уже реализован и минимально исполняется, однако рабочий прикладной маршрут продолжает использовать старый API.

Наличие отдельного теста нового класса снижает неопределённость вокруг самой реализации клиента, но не доказывает успешную миграцию системы. Для этого нужен тест, проходящий через ту же границу, которой пользуется прикладной код.

Пометка преподавателю. Откройте файл `tests/test_rag.py` и перейдите к тесту `test_checkout_uses_legacy_gateway`.

Этот тест фиксирует текущее наблюдаемое поведение через верхний уровень сценария. Он вызывает оформление заказа и проверяет, какой gateway оказался в результате платежа. После переключения сервиса на новую версию именно эта проверка должна измениться, потому что старое значение перестанет соответствовать цели миграции.

Название теста тоже является частью документации поведения. Если оставить имя, утверждающее использование legasy-шлюза, но поменять ожидаемое значение на новую версию, тест начнёт противоречить сам себе. План должен предусмотреть изменение смысла теста, а не только его литерала.

Пометка преподавателю. В том же файле перейдите к тесту `test_auxiliary_demo_modules`.

Здесь новый gateway проверяется напрямую, а имя текущего провайдера проверяется через сервис. Эти две проверки отвечают на разные вопросы. Прямой вызов подтверждает работоспособность нового API как отдельного объекта, а сервисная проверка показывает, какая реализация включена в приложение.

После миграции прямой тест новой версии может остаться без изменения. Ожидание от `get_payment_provider_name` должно отражать новый выбор сервиса. Такое разделение помогает не переписывать тесты целиком и сохраняет проверяемую связь между причиной изменения и конкретным ожиданием.

Отличаем пакетную зависимость от зависимости между модулями

Пометка преподавателю. Откройте файл `requirements.txt` в корне проекта.

Файл зависимостей описывает внешние Python-пакеты, необходимые приложению. Платёжные классы находятся внутри самого репозитория, поэтому их миграция не требует автоматически менять версию установленного пакета. Это другой тип зависимости, который определяется импортами между локальными модулями.

Я проверяю `requirements.txt`, чтобы явно исключить лишнюю область работы. ИИ нередко связывает слово «миграция» с обновлением библиотеки, хотя в текущем проекте целевой API уже лежит рядом со старым. Доказательство отсутствия внешней пакетной миграции позволяет не добавлять ненужные действия в план.

Фиксируем исходное поведение тестом

Пометка преподавателю. В терминале запустите только тестовый класс платёжного демонстрационного сценария.

```
.\.venv\Scripts\python.exe -m unittest tests.test_rag.DemoCodebaseTests -v
```

До изменения кода я получаю исходную точку сравнения. В текущем состоянии проходят проверка legasy-маршрута и прямой вызов нового gateway. Этот результат не означает, что миграция уже выполнена; он доказывает, что репозиторий находится в согласованном исходном состоянии.

После будущего изменения я запущу ту же команду. Сопоставимость проверки важнее количества тестов, потому что одинаковый маршрут до и после миграции позволяет связать новый результат именно с изменением API.

Анализируем происхождение решения через Git

Пометка преподавателю. В терминале покажите историю файла сервисного слоя.

```
git log --follow --oneline -- demo_codebase/payments/service.py
```

Ключ `--follow` нужен, чтобы Git продолжал искать историю через переименование файла, если оно существовало. В текущем репозитории сервис был добавлен одним учебным коммитом и после этого отдельно не менялся. Поэтому история не содержит постепенного перехода между версиями API.

Я использую этот факт как ограничение анализа. Git может показать происхождение текущего решения, но не может объяснить мотив, который никогда не был зафиксирован отдельным изменением. Если история короткая, я не придумываю эволюцию архитектуры, а опираюсь на текущие связи и тесты.

Пометка преподавателю. В терминале покажите статистику последнего изменения `service.py`.

```
git show --stat --oneline (git log -1 --format=%H -- demo_codebase/payments/service.py) -- demo_codebase/payments/service.py
```

Статистика показывает контекст появления файла без вывода полного patch. Для навигации этого достаточно, чтобы понять масштаб исходного изменения и связать сервис с этапом развития проекта. Если нужен смысл конкретной строки, тогда я перехожу к diff, но не начинаю с большого вывода всего коммита.

История файла и история архитектурного решения не всегда совпадают. Один коммит мог добавить сразу несколько модулей, поэтому сообщение коммита описывает более широкую учебную работу. Я не переношу смысл всего коммита на одну строку кода без дополнительной проверки.

Пометка преподавателю. В терминале выполните blame для сервисного слоя.

```
git blame --date=short -- demo_codebase/payments/service.py
```

`git blame` связывает текущие строки с последним коммитом, который их изменил. В нашем случае весь файл указывает на один источник, что согласуется с короткой историей. Эта команда отвечает на вопрос о происхождении текущего текста, но не определяет виновника проблемы и не доказывает правильность решения.

Я использую blame как переход к контексту коммита. Если строка появилась вместе с большим набором учебных файлов, следующим источником смысла становятся соседние изменения и тесты из того же коммита. Авторство само по себе не является архитектурным объяснением.

Пометка преподавателю. В терминале найдите коммит, изменивший количество вхождений старого метода.

```
git log -S "charge_legacy" --oneline -- demo_codebase
```

Поиск `-S` работает по истории содержимого и находит изменения, где количество точных вхождений строки поменялось. Для нашего проекта он приводит к коммиту, в котором legacy-вызов появился. Это отличается от обычного поиска по текущему рабочему дереву: `rg` отвечает, где строка находится сейчас, а `git log -S` отвечает, когда она вошла в историю.

Такой поиск особенно полезен после реальной миграции. Если старый вызов исчезнет, история сохранит как его появление, так и удаление. Тогда следующий разработчик сможет восстановить границу перехода без чтения всей последовательности коммитов.

Формируем границу миграции

На основании проверенных файлов я локализую изменение в `demo_codebase/payments/service.py`. Именно сервис импортирует legacy-класс, создаёт его, вызывает старый метод и сообщает имя текущего провайдера. Эти проявления относятся к одному архитектурному решению и должны измениться согласованно.

Файл `gateway.py` уже содержит целевой API, поэтому план не требует переписывать его только ради активности. Перед реализацией достаточно подтвердить, что метод новой версии принимает данные, которые уже передаёт сервис, и возвращает форму результата, которую способен принять вызывающий код.

Файл `checkout.py` находится внутри радиуса проверки, но не внутри обязательной области редактирования. Он зависит от функции сервиса, а не от класса gateway. Если контракт `process_payment` сохраняется, изменение checkout будет признаком того, что миграция вышла за намеченную границу.

Внешне наблюдаемое поле `payment.gateway` изменится с legacy-значения на значение новой версии. Это ожидаемый результат, а не случайная регрессия. План должен прямо зафиксировать его, чтобы обновление теста не выглядело попыткой скрыть ошибку.

Строковое совпадение в `tests/test_dev_tools.py` не входит в рабочий маршрут. Я оставляю решение по нему за назначением теста поискового инструмента. Если тест должен продолжать искать legacy-термин, строка сохраняется; если учебная цель меняется на поиск нового API, корректируется сам сценарий теста, а не только найденное слово.

Просим ИИ подготовить черновик плана

Пометка преподавателю. В чате ИИ-ассистента отправьте следующий запрос.

Подготовь план миграции платёжного сервиса с LegacyPaymentGateway на PaymentGatewayV2. Используй только подтверждённые файлы текущего репозитория, не изменяй код, отдели обязательные изменения от файлов, которые нужны только для проверки, и для каждого вывода укажи источник в проекте.

Я прошу ассистента подготовить план только после ручной проверки маршрута. Теперь у модели есть узкая область и проверенные опорные точки, поэтому вероятность выдуманного файла или лишнего рефакторинга ниже. Ссылки на источники позволяют сразу обнаружить вывод, который не подтверждается репозиторием.

Хороший план описывает не движения курсора, а переход системы из одного проверяемого состояния в другое. В нём должна быть понятна точка замены, ожидаемое изменение поведения и способ доказать результат. Команды реализации могут различаться, но архитектурная граница должна оставаться обоснованной.

Оформляем проверенный план миграции

Пометка преподавателю. Создайте в корне проекта файл `migration-plan.md` и откройте его в редакторе.

Я оформляю план обычными разделами и связными абзацами. Такой документ можно проверить до начала реализации, а после завершения сопоставить с фактическим diff. Он не должен превращаться в копию ответа ИИ без проверки источников.

Цель миграции

Я фиксирую, что рабочий маршрут оплаты должен перейти с `LegacyPaymentGateway` на `PaymentGatewayV2`. Точкой переключения является функция `process_payment`, потому что через неё проходит прикладной вызов из `checkout.py`.

Обязательная область изменения

Я указываю `demo_codebase/payments/service.py` как файл реализации миграции. В нём целевой клиент должен заменить legacy-клиент, вызов должен использовать метод нового API, а функция имени провайдера должна перестать сообщать legacy-значение.

Сохраняемая граница

Я записываю, что сигнатура `process_payment` и структура результата оплаты сохраняются. Благодаря этому `demo_codebase/orders/checkout.py` не требует изменения. Поле, обозначающее фактически использованный gateway, намеренно получает значение новой версии.

Проверка результата

Я связываю миграцию с `DemoCodebaseTests` из `tests/test_rag.py`. Тест верхнего маршрута должен подтверждать новую версию gateway, а проверка имени провайдера должна соответствовать новому выбору сервиса. Прямой тест `PaymentGatewayV2` остаётся подтверждением целевого API, но не заменяет проверку маршрута через checkout.

Контроль области

Я заранее отмечаю, что наличие legacy-строки в тесте поискового инструмента не доказывает рабочую зависимость. Решение об этом совпадении принимается отдельно по назначению `test_dev_tools.py`. Внешние зависимости проекта менять не требуется, поскольку обе реализации gateway уже находятся в репозитории.

Условие завершения

Я считаю план выполненным, когда целевой тестовый класс проходит после изменения, поиск рабочего кода больше не находит вызов `charge_legacy`, а diff остаётся внутри обоснованной архитектурной границы. История Git должна сохранять понятное описание перехода, но конкретная стратегия фиксации изменений определяется правилами проекта.

Возврат к исходному состоянию

Я предусматриваю простой откат на уровне сервисного адаптера. Поскольку checkout не изменяется, возврат legacy-клиента не требует обратной перестройки прикладного маршрута. Параллельная поддержка двух клиентов в текущем учебном сценарии не нужна, пока отдельное требование совместимости не появилось в контракте.

Проверяем план по репозиторию

Пометка преподавателю. В терминале повторите поиск старого метода перед началом реализации.

```
rg -n "charge_legacy" demo_codebase tests
```

До миграции рабочий вызов должен оставаться видимым в `service.py`. Я сохраняю этот результат как исходное доказательство. После реализации та же команда позволит отличить оставшееся объявление legacy-метода от рабочего вызова и от тестового текста.

План не обещает удалить любую строку со словом `legacy`. Его задача — исключить старый API из рабочего платёжного маршрута и обновить наблюдаемое поведение. Это точнее, чем требование получить пустой результат глобального поиска, потому что репозиторий может намеренно хранить совместимый класс, исторический тест или документацию.

Пометка преподавателю. В редакторе сравните подготовленный `migration-plan.md` с открытым файлом `demo_codebase/payments/service.py`.

Я проверяю, что каждому обязательному изменению из плана соответствует реальное место в сервисе, а каждый упомянутый тест связан с наблюдаемым результатом этого сервиса. Если в плане появляется файл без подтверждённой зависимости, я удаляю его из области или отдельно обосновываю необходимость.

На этом практика заканчивается до изменения исходного кода. Мы получили проверяемый план миграции, построенный не по догадке модели, а по текущим файлам, направлению импортов, тестовому контракту и истории Git. Следующий этап реализации можно начинать только с этой зафиксированной границы.